Lab 2: Energy Efficient Displays

Your Name
Student ID

July 5, 2026

1 Introduction

2 Background

2.1 Spiking Neural Networks

2.1.1 Hard vs Soft Reset

2.2 Spiking Convolutional Neural Networks

Spiking Convolutional Neural Networks are SNNs with convolutional connections. One of the best networks that achieves 99.1% accuracy on the MNIST dataset is a Spiking ConvNet, which is trained as an ANN and then transformed into a Spiking ConvNet in Diehl et al. [1]. This work also has the Spiking fully connected network version, which has a 98.6% accuracy with the MNIST dataset. These results show us that the convolutional connectivity achieves a higher accuracy. Thus, making the convolutional support for the C code generation crucial.

The C code implementation is done with the

2.2.1 Backward Pass

2.2.2 Forward Pass (Inference)

3 Related Work

4 Methodology and Pipeline Overview

5 Optimized C Code Generation

The basis is the nir_to_c_generator.py script, which generates optimized C code for Cortex-M7 ARM MCUs. In the previous version, the capabilities include linear and recurrent connected neural networks with a hard-reset (Reset-to-value) mechanism. With my work, the soft-reset (reset-by-subtract) mechanism, convolutional (conv2d) support, and sparsity optimization were introduced.

TODO: Rephrase the sparsity part after implement it.

5.1 Soft Reset

The first thing to implement is Soft Reset (Reset-by-subtraction), The previous work had only the hard reset (Reset-to-value) and lacked the soft reset mechanism.

First of all, the soft-reset mechanism introduced here follows the same approach as in Eshraghian et al. [2]. Also, it is implemented in the `snnTorch` Python library in the same

way, enabling the Python golden reference to match this work. The mathematical representation of the soft-reset mechanism is given in the last part of Equation 1.

$$U[t] = \underbrace{\beta U[t-1]}_{\text{decay}} + \underbrace{WX[t]}_{\text{input}} - \underbrace{S_{\text{out}}[t-1]\theta}_{\text{reset}} \tag{1}$$

In Figure 1, the soft-reset implementation in C is given. First, the current timestep membrane update is being calculated with vectorized ARM functions. After the membrane update, the spike check is done. While comparing the potential to the threshold, if there is a spike, the threshold value is saved to be subtracted at the next timestep.

```c
// Membrane Update
q15_t temp1[num_neurons], temp2[num_neurons];

arm_mult_q15(membrane_potentials, decay_factors, temp1, num_neurons);
arm_add_q15(temp1, weighted_inputs, temp2, num_neurons);
arm_sub_q15(temp2, reset_values, membrane_potentials, num_neurons);

// Spike check, then store reset_value for the NEXT step
for (uint16_t i = 0; i < num_neurons; i++) {
    if (membrane_potentials[i] > thresholds[i]) {
        output_spikes[i] = 1;
        neurons[i].reset_value = thresholds[i]; // subtract next step
    } else {
        output_spikes[i] = 0;
        neurons[i].reset_value = 0;
    }
    neurons[i].membrane_potential = membrane_potentials[i];
}
```

Figure 1: Soft-Reset Implementation in C

5.2 Convolutional Support

First of all a basic convolutional support added. Its pseudo-code shown in the Figure **??**. The convolution calculation is done with a sliding window in a deductive manner, from the output neuron to the input neuron. For every output neuron, the sliding window is being calculated first

5.3 Sparsity-Aware Inference

6 Embedded Deployment

6.1 CMSIS-DSP and CMSIS-NN

6.2 UART DMA

6.3 Quad-SPI Flash Chips

6.4

7 Model Training and Testing

7.1 Braille

7.2 ST-MNIST

8 Experimental Results

9 Discussion

[2]

A Appendix

References

[1] Peter U. Diehl, Daniel Neil, Jonathan Binas, Matthew Cook, Shih-Chii Liu, and Michael Pfeiffer. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8, 2015.

[2] Jason K. Eshraghian, Max Ward, Emre O. Neftci, Xinxin Wang, Gregor Lenz, Girish Dwivedi, Mohammed Bennamoun, Doo Seok Jeong, and Wei D. Lu. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, 2023. snnTorch reference paper.


```
Example Code
```

Figure 2: Transformation function definitons

Figure 3: Pareto graph of different image transformation techniques

Table 1: Power saving statistics for each transformation

Transform	**Mean PS (%)**	**Min PS (%)**	**Max PS (%)**
Bright scale	17.18	16.85	18.46
Histogram	-5.53	-135.09	30.56
Hungry blue	6.43	2.78	20.24